

1 Memory Abstraction

1.1 Memory Management

Random Access Memory (RAM)

- **Volatile** memory: data is lost when power is turned off
- **Byte-addressable**: each byte has a unique address
- **Directly accessible** by CPU: CPU can read/write to RAM without any intermediate steps
- **Contiguous memory region**: consists an interval of **consecutive** addresses

In a process, there are generally two types of data:

- **Transient Data**: Data that is valid only for a limited duration
- **Persistent Data**: Data that is valid for the entire duration of the process unless explicitly deallocated

The OS **manages the memory space** of processes:

- Allocate memory space for new processes
- Manage memory space for processes
- Protect memory space of processes from each other
- Provides memory related system calls
- Manage memory space for internal OS data structures

Without memory abstraction, physical addresses are directly used.

Memory access is straightforward, but

- Processes might not have **contiguous memory space**
- Memory addresses may not be **consistent across runs** (and do not begin at 0)
- **Hard to protect memory space** of processes from each other, as they can directly access physical addresses of other processes

Address Relocation: Map all memory references of a process to the actual physical addresses. **e.g. Add a constant offset** to all memory references. Results in **slow loading time**, and memory references are **difficult to distinguish** from integer constants (in binary form).

Base and Limit Registers: Special registers in **memory context** storing the offset of all memory references (starting address) and the range (length of memory space). Incurs an **additional addition and comparison** for each memory access (even for instruction retrieval).

Logical Address

Having a **logical interface** for memory accesses to processes, where each process has its own **logical address space that is mapped to the physical address space** by the OS.

1.2 Contiguous Memory Management

Assumptions:

- Processes (and data) must be loaded into memory for execution (**Store Memory concept**)
- Instructions operate solely on registers except for load and store that access memory (**Load-Store Memory**)
- Processes require **contiguous memory space** (Memory Partition)
- Physical memory is **large enough** to hold all processes
 - Removing terminated processes
 - Swapping out blocked processes to secondary storage

Partition allocation schemes

Fixed size partitioning: Memory is divided into fixed number of partitions. Each partition can hold one process.

- **Easy to manage**
- **Fast** to allocate and deallocate
- Partition size must be **large enough** to hold the largest process
- Can lead to **internal fragmentation** (wasted space within partitions)

Variable size partitioning: Memory is allocated dynamically based on the size of the process.

- More **efficient** use of memory
- More **flexible**, able to accommodate processes of varying sizes
- Eliminates **internal fragmentation**
- Can lead to **external fragmentation** (large number of "holes", each too small for new processes)
- **More information** to manage in OS
- **Slower** to allocate and deallocate

1.3 Dynamic Partitioning

Allocation

A list of partitions and holes is maintained. When a partition of size N is needed, OS searches for a suitable hole of size $M > N$:

- **First-Fit**: Allocate the first hole that is large enough.
- **Best-Fit**: Allocate the smallest hole that is large enough.
- **Worst-Fit**: Allocate the largest hole.

and splits the hole into N and $M - N$.

Deallocation

When a process terminates, the occupied partition is freed and becomes a hole. OS can reduce fragmentation by:

Merging: Combine adjacent free partitions into a single larger hole.

Compaction: Move partitions to consolidate holes and eliminate fragmentation. **Expensive operation**, cannot be invoked too frequently.